Study Project – Phase 3 Document (Implementation Readiness & Validation)

Cover Page

Course Title:	Study Project
Project Title:	Operational Complexity Meter (OCM): Quantifying Operational Complexity in Distributed Systems
Group Number:	1
Student Name(s):	Aabid Ali Sofi
Student ID(s):	2023EBCS041
Project Advisor / Supervisor:	Preethy P Johny
Date of Submission:	March 2, 2026

1. Introduction

1.1. Purpose of Phase 3

Phase 3 demonstrates that the Operational Complexity Meter (OCM) has reached full implementation readiness. The objectives of this phase are:

- Demonstrating implementation readiness by delivering a feature-complete system that computes all six operational complexity metrics.
- Validating design choices through automated testing, evidence traceability, and end-to-end pipeline verification.
- Assessing system reliability, limitations, and future potential based on real-world testing against Kubernetes repositories.

1.2. Summary of Work Completed So Far

Phase 1 — Problem Definition & Planning

- Identified the gap: no tool exists that quantifies *operational* complexity (deployment, configuration, dependency management) as distinct from code complexity.
- Defined the six metrics (CSA, DD, DB, CV, FE, CDR) and the composite OCM scoring model.
- Planned the single-binary Go architecture with embedded SQLite and web dashboard.

Phase 2 — Design & Proof of Concept

- Delivered a working PoC implementing two of six metrics (CSA and DD).
- Established the end-to-end pipeline: YAML parsing → metric computation → SQLite persistence → REST API → web dashboard.
- Defined the database schema, functional requirements (FR1–FR10), and modular package structure.
- Validated feasibility with a demo against a sample Kubernetes repository.

2. Implementation Overview

2.1. Implementation Status

All modules planned during Phase 2 have been **fully implemented**. The system is feature-complete with respect to the six-metric scope defined in Phase 1.

Fully Implemented Modules

Module	Description
`model`	Metric types, default weights (equal 1/6 each), data structures for services, metric points, scores, evidence, and analysis results.
`parser`	Kubernetes YAML parser — extracts CSA facts (env vars, ports, resources, replicas, spec keys), dependency relationships (7 suffix patterns, hostname:port regex, `.svc.cluster.local` FQDN), and Failure Exposure signals (5 detection rules). Handles Deployment, StatefulSet, DaemonSet, Job, CronJob, Service, and Ingress kinds.
`pipeline`	Full orchestration — discovery, service identification (`dir`/`manifest` strategies), all six metric computations, min-max normalization, and weighted composite scoring.
`gitlog`	Git history extraction for Change Volatility — parses `git log` output, filters by config file extensions, maps files to services, computes per-

	service commit counts within a configurable time window (default 30 days).
`storage`	SQLite persistence — schema migration, transactional run saves (upsert services, insert metrics/scores/evidence), time-series queries, evidence retrieval.
`api`	REST API — 6 endpoints for health, services, overview aggregates, score series, metric series, and evidence. CORS middleware for browser access.
`dashboard`	Embedded web UI — `go:embed` static assets, Stripe-inspired dark theme, 4 canvas-based chart types, responsive layout, evidence modal.
`cmd/ocm`	CLI entry point — 7 flags, orchestrates the full startup sequence (parse ⟶ analyze ⟶ persist ⟶ serve).

Partially or Unimplemented: None. All planned components are complete.

2.2. Implemented Features

The following features have been implemented end-to-end:

Core Metric Computation (6 metrics):

1. **CSA (Configuration Surface Area)** — Count of configurable knobs: environment variables, container ports, resource limits/requests, replicas, and spec-level keys.
2. **DD (Dependency Depth)** — Longest path in the SCC-condensed dependency DAG, computed using Tarjan's algorithm for deterministic cycle handling.
3. **DB (Dependency Breadth)** — Sum of in-degree and out-degree in the service dependency graph.
4. **CV (Change Volatility)** — Git commits touching service configuration files within a configurable time window (default 30 days).
5. **FE (Failure Exposure)** — Externally exposed endpoints detected via 5 rules: LoadBalancer/NodePort services, Ingress resources, hostPort bindings, and external URLs in environment variables.
6. **CDR (Configuration Drift Risk)** — Environment-specific overrides counted across recognized environment directories (dev, staging, prod, qa, test, uat, preview, canary, local).

Composite Scoring:

- Min-max normalization per metric across the service cohort:

$$\text{norm} = \frac{v - \min}{\max - \min}$$

- When $\max = \min$, normalized value defaults to 0.
- Composite OCM score:

$$\text{OCM} = \sum_{m \in M} w_m \cdot \text{norm}_m$$

  where $w_m = 1/6$ for all six metrics.

Data Handling and Persistence:

- SQLite database with 4 tables and 3 indexes, foreign key enforcement.
- Transactional writes — each analysis run is persisted atomically.
- Time-series storage for trend analysis across multiple runs.
- Evidence items linked to specific metrics, services, and source files.

User Interaction Flows:

- CLI with 7 configurable flags for repository path, database, host/port, service key strategy, CV window, and JSON output.
- Web dashboard with two views: **Overview** (repo-wide aggregates) and **Services** (comparison table).
- Per-service detail: metric tiles with sparklines, trend chart, radar chart, bar chart.
- Evidence drill-down modal showing Component, Key, Value, Manifest, and Source columns.
- Responsive layout with mobile sidebar toggle.

Integration:

- Git integration for CV metric (best-effort — gracefully degrades when Git is unavailable).
- Filesystem-based environment detection for CDR metric.
- CORS middleware for cross-origin browser access.

3. System Validation and Testing

3.1. Testing Strategy

The project employs a layered testing approach:

- **Unit testing:** Isolated tests for individual functions (normalization, dependency extraction, git log parsing, service name normalization).
- **Integration testing:** End-to-end pipeline tests that create temporary Kubernetes YAML repositories, run the full analysis pipeline, and verify all six metrics are computed with correct values.
- **Persistence testing:** Round-trip tests that save analysis runs to an in-memory SQLite database and verify retrieval of all metric types and evidence items.

All tests use Go's standard `testing` package and can be run with `go test ./...`.

3.2. Test Cases and Results

The test suite consists of **32 test functions** across 4 packages.

Parser Tests (16 tests)

Test	Expected Behavior	Observed	Status
FE: Service LoadBalancer	2 ports ⟶ 2 exposed endpoints	2 endpoints, 2 evidence items	Pass
FE: Ingress	2 rules, 3 paths ⟶ 3 endpoints	3 exposed endpoints	Pass
FE: ExternalName	1 external integration	component = `external_integration`	Pass
FE: External URL	Only external URLs counted	1 external URL (internal skipped)	Pass
FE: NodePort	1 port ⟶ 1 endpoint	1 exposed endpoint	Pass
FE: ClusterIP	No exposure	0 endpoints	Pass
Dep: Service suffix	`DB_SERVICE=postgres` ⟶ dep	[postgres, redis]	Pass
Dep: Addr suffix	Google Boutique pattern	[cartservice, productcatalogservice]	Pass
Dep: Host suffix	`DB_HOST=postgres:5432`	[postgres, redis]	Pass
Dep: URL suffix	`API_URL=http://api-gateway:8080`	[api-gateway]	Pass

Dep: Hostname:port	Bare `mongo=user-db:27017`	[user-db]	Pass
Dep: Cluster FQDN	`.svc.cluster.local`	[zipkin]	Pass
Dep: Plain strings	No deps from plain values	0 dependencies	Pass
Dep: Numeric rejected	Port numbers not deps	1 dep (postgres only)	Pass
Normalize: Numeric	`"5432"` → `""`, `"redis"` → `"redis"`	Correct filtering	Pass
Dep: CronJob	CronJob containers parsed	[grafana, prometheus]	Pass

Pipeline Tests (8 tests)

Test	Expected Behavior	Observed	Status
DD: Cycle handling	SCC condensation: a↔b cycle	dd[a]=1, dd[b]=1, dd[c]=0	Pass
Normalize: max=min	Same values → 0	Both normalize to 0	Pass
Pipeline: Run smoke	Produces valid score	Score in [0,1]	Pass
DB: In/out degree	a→{b,c}, b→{c}	db[a]=2, db[b]=2, db[c]=2	Pass
DB: No dependencies	Isolated services	db = 0 for both	Pass
DB: Self-dep ignored	a→{a,b}, self-dep skipped	db[a]=1, db[b]=1	Pass
All 6 metrics (integration)	Temp repo with 2 services	All metrics present, correct ranges	Pass
All metrics in composite	Manual data, full pipeline	Normalized values in [0,1]	Pass

Git Log Tests (6 tests)

Test	Expected Behavior	Observed	Status
Multiple commits	2 commits parsed correctly	Correct hashes, timestamps, files	Pass
Empty output	0 commits returned	Empty slice	Pass
Single commit, no files	1 commit, 0 files	Correct	Pass
Malformed timestamp	Hash parsed, time is zero	Graceful degradation	Pass
Hash only	Hash parsed without timestamp	Correct	Pass
Options defaults	30-day window, extensions set	Defaults applied	Pass

Storage Tests (2 tests)

Test	Expected Behavior	Observed	Status
Save/retrieve all metrics	Round-trip for all 6 types	All evidence items verified	Pass

Empty source path	DD/DB/CV/CDR evidence saves	Successfully persisted	Pass

3.3. Validation Summary

- **Requirements coverage:** All 10 functional requirements from Phase 2 (FR1–FR10) are satisfied. Additionally, four new metrics (DB, CV, FE, CDR) extend the system beyond the original MVP scope.
- **Evidence traceability:** Every metric value is backed by queryable evidence items that trace back to specific YAML files, manifest kinds, and configuration keys.
- **Deviations:** No deviations from expected behavior. The system handles edge cases (cycles in dependency graphs, missing Git, numeric values, CronJob manifests) as designed.

4. Performance and Reliability Analysis

Responsiveness:

- Analysis of small-to-medium repositories (10–50 services) completes in under 5 seconds.
- The Git log extraction (`git log --since=...`) adds a few seconds depending on repository history depth.
- API responses and dashboard rendering are sub-second for typical workloads.

Stability:

- The system runs as a single Go binary with no background threads or goroutine leaks.
- SQLite transactions ensure atomic writes — partial analysis runs are not persisted.
- The HTTP server uses a 5-second `ReadHeaderTimeout` to prevent slow client attacks.
- The dashboard handles API errors gracefully with status indicator dots (green/red).

Resource Usage:

- Memory footprint is minimal: YAML files are parsed one document at a time, and the dependency graph is built in-memory only for the current run.
- SQLite database size grows linearly with the number of runs and services (50KB per run for a 10-service repo).
- The binary size is approximately 15MB (including embedded static assets and the pure-Go SQLite driver).

Performance Bottlenecks:

- The `git log` subprocess is the slowest component for repositories with extensive commit histories. The configurable `--cv-window` flag mitigates this by limiting the lookback window.
- CDR environment detection relies on directory name heuristics, which may produce false positives for unconventional directory structures.

5. Risk Analysis and Mitigation Review

5.1. Identified Risks (Revisited)

The following risks were identified during Phase 1 and Phase 2:

Risk	Category	Description
R1	Service identification ambiguity	Different directory structures may yield inconsistent service names.
R2	Dependency extraction uncertainty	Heuristic-based dependency detection may miss or falsely identify dependencies.

R3	Metric validity vs heuristics	Metrics based on heuristics may not accurately reflect true operational complexity.
R4	Scope creep	Expanding beyond the six-metric scope during implementation.
R5	External tool dependency	CV metric depends on Git being available in the environment.
R6	Scalability for large repos	Memory and time costs for repositories with hundreds of services.

5.2. Mitigation Effectiveness

Risk	Mitigation Applied	Effectiveness	Status
R1	Two strategies (`dir` and `manifest`) configurable via `--service-key` flag.	Effective	Mitigated
R2	Seven suffix patterns, hostname:port regex, FQDN detection, and CronJob support. 16 parser tests validate extraction.	Effective	Mitigated
R3	Full evidence traceability — every metric value links to specific YAML keys, files, and manifests. Users can audit why a score is what it is.	Effective	Mitigated
R4	Strict adherence to the six-metric model. No features added outside the defined scope.	Fully effective	Mitigated
R5	Best-effort approach: if `git` is not in PATH, CV returns zero and the pipeline continues without error.	Effective	Mitigated
R6	Sequential file processing avoids memory spikes. Not tested at extreme scale (500+ services).	Partially effective	Remaining

6. Limitations and Constraints

Current Limitations:

- **No Helm support:** The parser operates on raw YAML files. Helm charts must be rendered (`helm template`) before analysis.
- **Heuristic dependency detection:** Dependencies are inferred from environment variable names and values. Services communicating through mechanisms not reflected in YAML (e.g., service mesh sidecars, message queues configured externally) are not detected.
- **Single-run model:** The analysis runs once on startup. There is no daemon mode, scheduled execution, or file-watching capability.
- **Localhost only:** No authentication or authorization. The dashboard and API are intended for local use only.
- **No multi-repo support:** Each run analyzes a single repository path. Cross-repository dependency analysis is not supported.

Assumptions:

- Kubernetes manifests follow standard API conventions (Deployment, Service, Ingress, etc.).

- Service names are stable and deterministic based on directory structure or manifest metadata.
- Git history is available in the analyzed repository for CV metric computation.
- Environment directories follow common naming conventions (dev, staging, prod, etc.).

Constraints:

- **Time:** Study project timeline limited advanced features such as CI/CD integration and multi-user access.
- **Technology:** Pure-Go SQLite driver (`modernc.org/sqlite`) trades some performance for zero CGO dependency, simplifying distribution.
- **Scope:** Only Kubernetes YAML is supported as input. Docker Compose, Terraform, and other IaC formats are excluded.

7. Future Enhancements and Scope Extension

Feature Enhancements:

- **Helm chart support:** Integrate `helm template` rendering to analyze parameterized charts directly.
- **Watch mode:** File-system watcher for continuous analysis as manifests change during development.
- **Custom metric weights:** Allow users to configure per-metric weights via CLI flags or a configuration file.
- **Multi-repository analysis:** Compare operational complexity across multiple repositories in a single dashboard.
- **CI/CD integration:** Run OCM as a CI pipeline step and fail builds that exceed an OCM threshold.

Performance Optimizations:

- **Incremental analysis:** Detect changed files since the last run and recompute only affected services.
- **Parallel parsing:** Use Go goroutines to parse YAML files concurrently for large repositories.
- **Database compaction:** Prune old time-series data to prevent unbounded SQLite growth.

Scalability Improvements:

- **Remote database support:** Replace or supplement SQLite with PostgreSQL for team-wide deployments.
- **Authentication:** Add token-based authentication for non-localhost usage.

Additional Use Cases:

- **Docker Compose and Terraform support:** Extend the parser to analyze other infrastructure-as-code formats.
- **Alerting:** Notify teams when a service's OCM score crosses a defined threshold.
- **Historical diff view:** Show metric changes between analysis runs in the dashboard.
- **Export:** Generate PDF or CSV reports of analysis results.

8. Learning Outcomes and Reflections

Technical Skills Gained:

- Designing and implementing a metrics engine with evidence traceability in Go.
- Working with graph algorithms (Tarjan's SCC, DAG longest path) for real-world dependency analysis.
- Building a complete application with embedded database, REST API, and web dashboard in a single binary — zero external runtime dependencies.
- Implementing canvas-based data visualizations (radar chart, sparklines, trend lines) without third-party charting libraries.
- Writing comprehensive tests for heuristic-based systems where edge cases are numerous and subtle.

Design and Problem-Solving Insights:

- **Evidence-first design:** Attaching evidence to every metric value proved essential for user trust. A complexity score is meaningless without the ability to explain *why* a service scored high.
- **Heuristic boundaries:** Dependency detection heuristics are never perfect, but combining multiple detection strategies (suffix patterns, regex, FQDN) with transparent evidence makes the system useful despite imperfection.
- **Min-max normalization trade-offs:** When all services have the same metric value, the normalized value defaults to 0. This is a deliberate design choice — a single-service repo should not appear "maximally complex" by default.

Challenges Faced and Lessons Learned:

- **YAML parsing complexity:** Kubernetes manifests have significant structural variation across resource kinds (Deployment vs CronJob vs Ingress). Supporting all common kinds required careful recursive traversal.
- **Cycle handling:** Circular dependencies in service graphs initially caused infinite loops during DD computation. Implementing Tarjan's SCC algorithm resolved this deterministically.
- **Git integration:** Parsing `git log` output is fragile. The structured `--format` flag mitigates this, and the best-effort approach ensures the system remains functional without Git.
- **Dashboard without frameworks:** Building responsive visualizations with vanilla Canvas and CSS required more effort than a framework-based approach but resulted in zero frontend build dependencies and a smaller bundle.

9. Final Deliverables

Deliverable	Description
Source Code	Complete Go codebase — 12 source files (3,800 lines) across 7 packages in the `internal/` and `cmd/` directories. Available at the project repository.
Compiled Binary	`ocm` — single executable, no external dependencies required at runtime.
Automated Test Suite	32 test functions covering parser, pipeline, git log, and storage modules. Run with `go test ./...`.
SQLite Database	`ocm.sqlite` — analysis results persisted for time-series tracking.
Web Dashboard	Embedded interactive dashboard with overview, service detail, charts, and evidence drill-down.
REST API	6 endpoints serving JSON data for services, metrics, scores, aggregates, and evidence.
Specification Documents	8 specification files in `specs/` covering data sources, parser, metric engine, composite scoring, persistence, API, and dashboard.
Phase 2 Document	Design and PoC document (`docs/PHASE_2.typ`).
Phase 3 Document	This document (`docs/PHASE_3.typ`).
Presentation Slides	Polylux/Typst presentation (`docs/presentation.typ`) with 19 slides covering the full system.

10. Conclusion

The Operational Complexity Meter (OCM) has been fully implemented as a single-binary Go tool that quantifies operational complexity in distributed systems deployed on Kubernetes. All six planned metrics — Configuration Surface Area, Dependency Depth, Dependency Breadth, Change Volatility, Failure

Exposure, and Configuration Drift Risk — are computed, normalized, and combined into a composite OCM score.

The system is validated by 32 automated tests covering metric computation, dependency graph algorithms, Git integration, and database persistence. Every metric value is backed by evidence items that trace to specific YAML files, manifest keys, and configuration parameters, ensuring full explainability.

The project has progressed from problem definition (Phase 1) through design and proof-of-concept (Phase 2) to a feature-complete implementation (Phase 3) that meets all defined functional and non-functional requirements. The modular architecture supports future extension to additional IaC formats, custom metric weights, and team-wide deployment.

The system is ready for evaluation and demonstration.

11. Supervisor Review and Approval

Advisor Feedback:

Supervisor Comments:

Recommendations:

Signature:

Date: